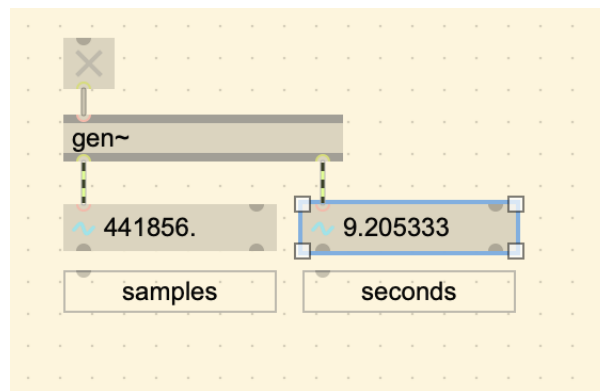


2. Modular (Arithmetic of) Time

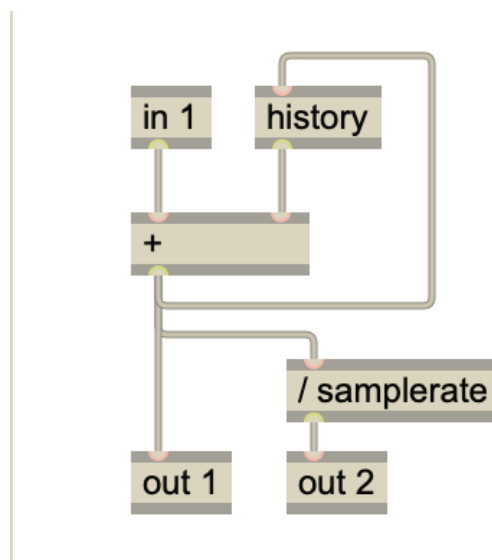
A simple counter

To create a simple counter, all we need are two objects:

- `[+]`
- `[history]`



Max MSP Patch



gen patch

In this patch, we are taking the one from the toggle. Essentially it is like having a `[1]` object going into the `[+]`, and accumulating 1 every sample frame. When DSP is on, `[in 1]` will send a 1, then it will add 1

+ 0, because the [history] object starts at 0, then sends it out to [out 1].

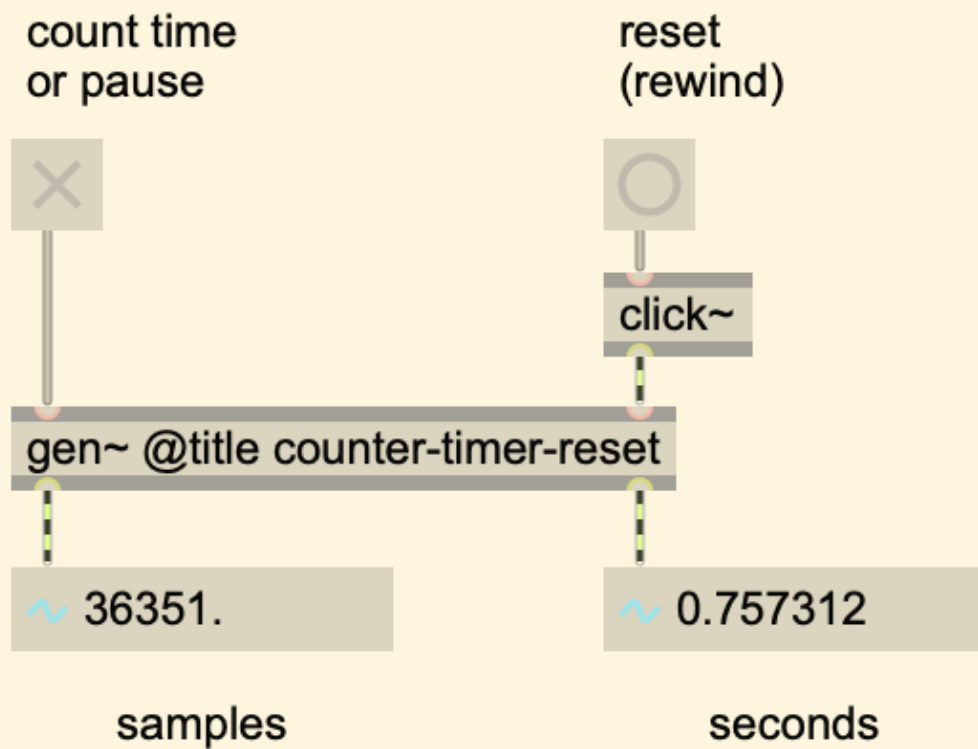
On the next sample frame, 1 comes from [in 1], and then it will do the addition of 1 + 1, because 1 was stored in the [history] object last sample frame. It will send the 2 to [out 1], and will repeat. On the 48000th sample frame (if at a sample rate of 48k), it will accumulate to 48,000, after 1 second.

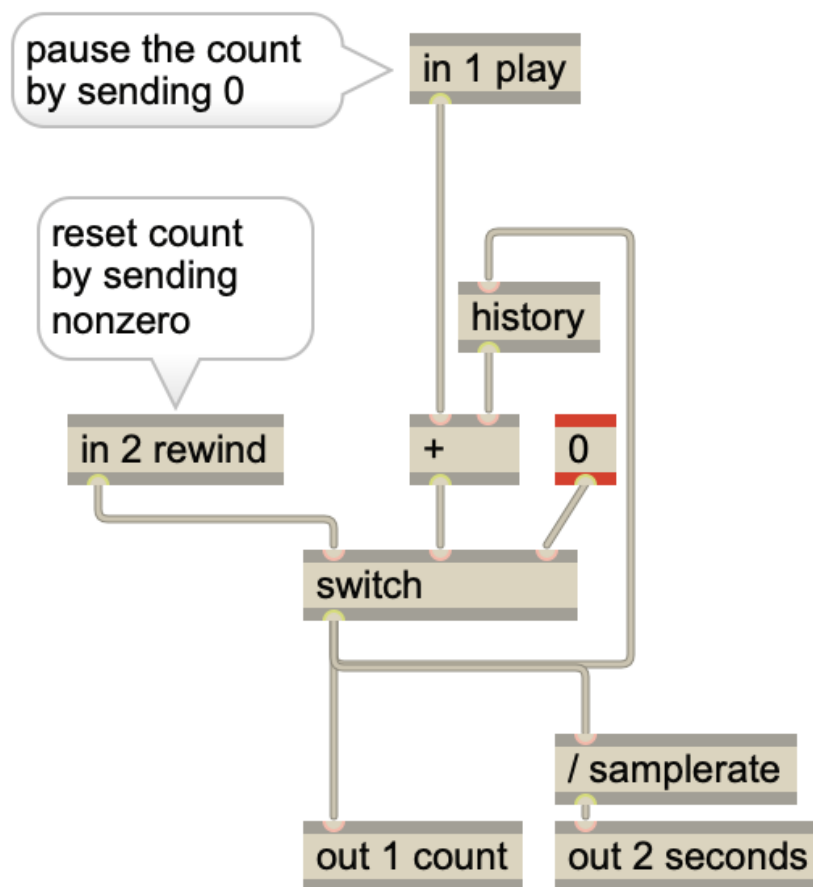
Since we know that 48k is one second, we can also get the time elapsed by dividing it by the sample rate. samplerate is like a constant with the value of the sample rate outside of the gen~ environment.

Adding a reset

To add a reset, all we have to do is send a [0] to the [history] object so it starts from 0 again. We can do that by using a [switch], so that it sends a [0] whenever it receives a trigger (a non-zero value). It just needs to receive a [0] for one sample to reset, so using a one sample trigger from a [click]~ to send a [0] works.

Rewinding a sample counter



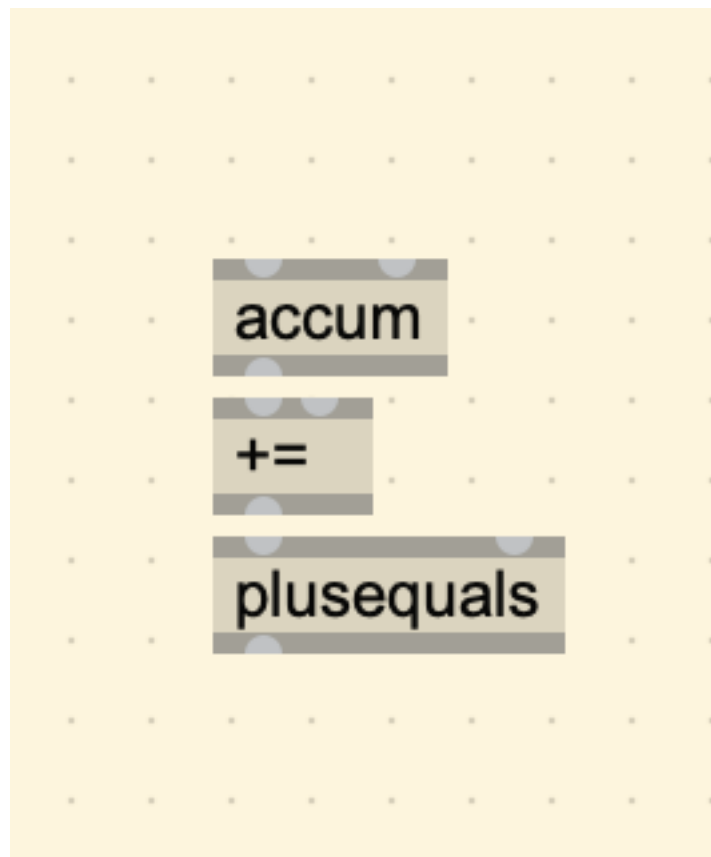


When the left inlet of the [switch] operator is false (0), then it will continue the count as normal. However if the [click~] is sent to [in 2], then it will send a true message (1), and the rightmost argument of the [switch] will go out the outlet, which is 0. This causes the [history] operator to store a 0, making it reset from there.

This counter functionality exists as an operator already called [accum], [+=], or [plusequals] These objects are the same thing.

Using [accum] to play a buffer

Modular Arithmetic



|200

